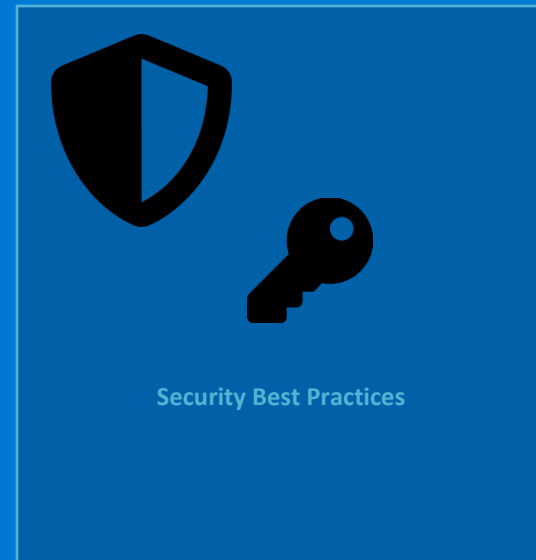


Sécurité avancée sur AKS

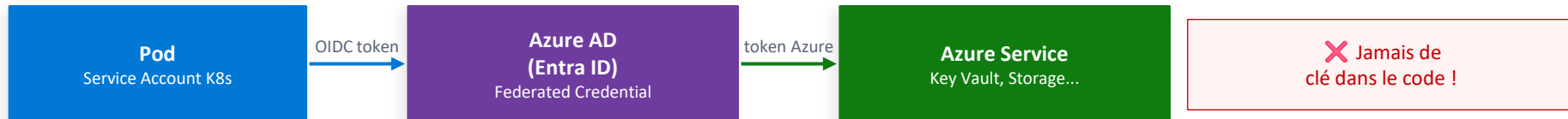
Workload Identity, Key Vault CSI, Pod Security et chiffrement



- 1 Workload Identity (identité des pods)
- 2 Azure Key Vault CSI Driver
- 3 Pod Security Admission (PSA)
- 4 Chiffrement en transit et au repos
- 5 Meilleures pratiques consolidées

Workload Identity — identité des pods

Permettre à un pod d'accéder à des services Azure sans secret dans le code



Activer Workload Identity sur AKS

```
# Activer OIDC Issuer + Workload Identity
az aks update -g myRG -n myAKS \
  --enable-oidc-issuer \
  --enable-workload-identity

# Créer la Managed Identity
az identity create -g myRG -n myapp-id

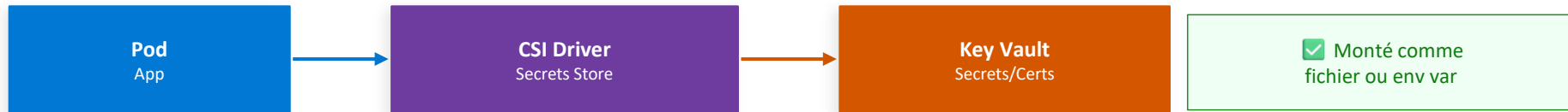
# Créer le federated credential
az identity federated-credential create \
  --name myapp-cred \
  --identity-name myapp-id \
  --issuer $OIDC_ISSUER \
  --subject system:serviceaccount:ns:sa
```

Avantages vs secrets

- ✓ Zéro secret dans le code ou les manifestes K8s
- ✓ Rotation automatique des tokens (durée de vie courte)
- ✓ Audit trail complet dans Azure AD
- ✓ Principe du moindre privilège — RBAC Azure précis
- ✓ Fonctionne en local (dev) via az login

Azure Key Vault CSI Driver

Injecter des secrets Key Vault dans les pods sans les stocker dans Kubernetes



```
apiVersion: secrets-store.csi.x-k8s.io/v1
kind: SecretProviderClass
metadata:
  name: kv-secrets
  namespace: $NAMESPACE_KV
spec:
  provider: azure
  parameters:
    usePodIdentity: "false"
    useVMManagedIdentity: "false"
    clientId: "$IDENTITY_CLIENT_ID"
    keyvaultName: "$KV_NAME"
    tenantId: "$TENANT_ID"
  objects: |
    array:
      - |
        objectName: db-password
        objectType: secret
      - |
        objectName: api-key
        objectType: secret
      - |
        objectName: app-config
        objectType: secret
```

Pourquoi Key Vault CSI ?

-  Secrets centralisés dans Key Vault — pas dans etcd K8s
-  Rotation automatique — le pod reçoit la nouvelle valeur sans redéploiement
-  Audit complet — chaque accès est logué dans Key Vault
-  Certificates X.509 montés directement dans le pod
-  Compatible Workload Identity — zéro secret de connexion à gérer

Pod Security Admission & Chiffrement

Restreindre les capacités des pods et protéger les données

Pod Security Admission (PSA)

Privileged

Aucune restriction — pour les nœuds système uniquement

Baseline

Empêche les élévations de privilèges connues

Restricted

Profil le plus sécurisé — best practice pour les apps

```
# Activer PSA sur un namespace
kubectl label ns production \
  pod-security.kubernetes.io/enforce=restricted
```

Chiffrement des données

At-rest (stockage)

Azure Disk/Files/Blob : AES-256 par défaut.
CMK via Azure Key Vault pour conformité RGPD/PCI.

etcd (secrets K8s)

Chiffrement at-rest d'etcd : activer --encryption-provider-config.
Sur AKS : géré par Azure, activé par défaut.

En transit (réseau)

TLS 1.2+ entre tous les composants K8s.
Ingress : cert-manager + Let's Encrypt (HTTPS).

```
# Activer CMK pour etcd
az aks update -g myRG -n myAKS \
  --enable-azure-keyvault-kms \
  --azure-keyvault-kms-key-id $KV_KEY_ID
```

Checklist sécurité minimale en production :

- ✓ Workload Identity activé
- ✓ Key Vault CSI pour les secrets
- ✓ PSA Restricted sur les namespaces apps
- ✓ Network Policies deny-all
- ✓ Private Cluster activé
- ✓ RBAC K8s + Azure RBAC

TP — Injecter un secret Key Vault dans un pod AKS

Durée : 40 min | Prérequis : cluster AKS, Key Vault créé, Workload Identity activé

1

Installer le Key Vault CSI add-on

```
az aks enable-addons -g myRG -n myAKS --addons azure-keyvault-secrets-provider
kubectl get pods -n kube-system | grep secrets-store
```

✓ Validation :

Pods secrets-store-csi-driver en Running

2

Créer un secret dans Key Vault

```
az keyvault secret set --vault-name myKV --name db-password --value 'myS3cr3t!'
az keyvault secret list --vault-name myKV -o table
```

✓ Validation :

Secret db-password visible dans Key Vault

3

Créer et appliquer le SecretProviderClass

```
kubectl apply -f secret-provider-class.yaml
kubectl describe secretproviderclass myapp-secrets
```

✓ Validation :

SecretProviderClass créé et validé

4

Déployer un pod et vérifier l'injection

```
kubectl apply -f pod-with-kv.yaml
kubectl exec -it myapp -- cat /mnt/secrets/db-password
# Doit afficher : myS3cr3t!
```

✓ Validation :

Secret monté en fichier — valeur lisible depuis le pod